INHA UNIVERSITY TASHKENT
FALL SEMESTER 2024

SOC 3010
OPERATING SYSTEM

CREDITS/HOURS PER WEEK : 3/3

COURSE TYPE : TECHNICAL CORE SEQUENCE

INSTRUCTOR
DR. A. R. NASEER
HEAD & PROFESSOR OF COMPUTER SCIENCE & ENGG

SOC 3010
OPERATING SYSTEM

WEEK 9 LECTURE2

PROCESSES

WISH YOU ALL THE BEST

PROCESSES

- A process is usually defined as an instance of a program in execution
- Processes are often called tasks or threads in the Linux source code.
- From the kernel's point of view, the purpose of a process is to act as an entity to which system resources (CPU time, memory, etc.) are allocated.

PROCESSES

- When a process is created, it is almost identical to its parent.
- It receives a (logical) copy of the parent's address space and executes the same code as the parent, beginning at the next instruction following the process creation system call.
- Although the parent and child may share the pages containing the program code (text), they have separate copies of the data (stack and heap), so that changes by the child to a memory location are invisible to the parent (and vice versa).

PROCESSES

- **Modern Unix systems support multithreaded applications - user programs having many relatively independent execution flows sharing a large portion of the application data structures.**
- **In such systems, a process is composed of several user threads (or simply threads), each of which represents an execution flow of the process.**
- **Nowadays, most multithreaded applications are written using standard sets of library functions called pthread (POSIX thread) libraries .**

PROCESSES

- Linux uses lightweight processes to offer better support for multithreaded applications.
- Basically, two lightweight processes may share some resources, like the address space, the open files, and so on.
- Whenever one of them modifies a shared resource, the other immediately sees the change.
- Of course, the two processes must synchronize themselves when accessing the shared resource.

PROCESSES

- **A straightforward way to implement multithreaded applications is to associate a lightweight process with each thread.**
- **In this way, the threads can access the same set of application data structures by simply sharing**
  - **the same memory address space,**
  - **the same set of open files, and so on;**
  - **at the same time, each thread can be scheduled independently by the kernel so that one may sleep while another remains runnable.**

PROCESSES

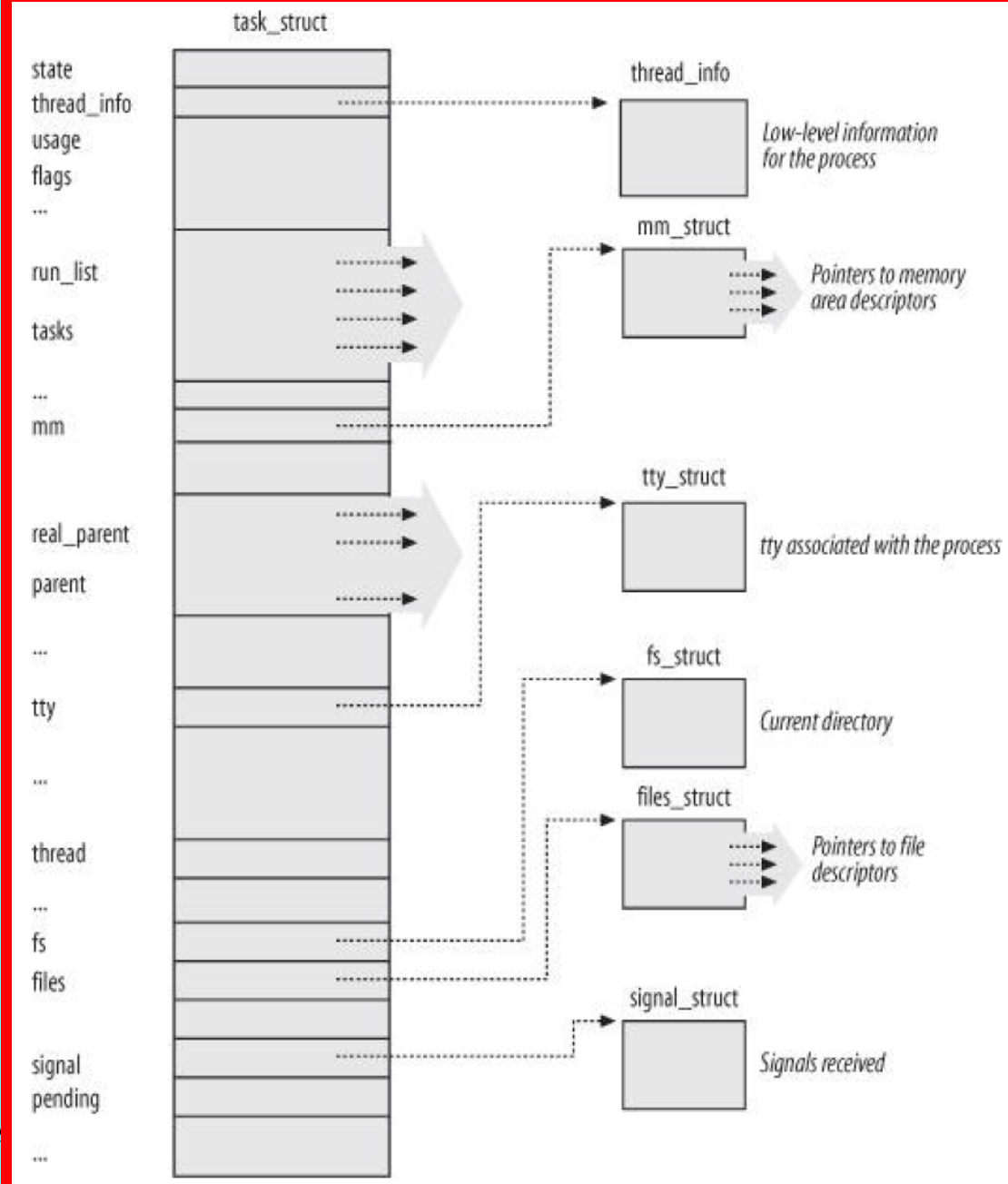
- Examples of POSIX-compliant pthread libraries that use Linux's lightweight processes are LinuxThreads, Native POSIX Thread Library (NPTL), and IBM's Next Generation Posix Threading Package (NGPT).
- POSIX-compliant multithreaded applications are best handled by kernels that support "thread groups ."
- In Linux a thread group is basically a set of lightweight processes that implement a multithreaded application and act as a whole with regards to some system calls such as getpid( ), kill( ), and _exit( )

PROCESS DESCRIPTOR

- To manage processes, the kernel must have a clear picture of what each process is doing.
- It must know, for instance, the process's priority, whether it is running on a CPU or blocked on an event, what address space has been assigned to it, which files it is allowed to address, and so on.
- This is the role of the process descriptor a task_struct type structure whose fields contain all the information related to a single process.
- In addition to a large number of fields containing process attributes, the process descriptor contains several pointers to other data structures that, in turn, contain pointers to other structures.

PROCESS DESCRIPTOR

- Figure describes the Linux process descriptor schematically.
- The kernel also defines the task_t data type to be equivalent to struct task_struct.
- The six data structures on the right side of the figure refer to specific resources owned by the process.



PROCESS STATES

- As its name implies, the state field of the process descriptor describes what is currently happening to the process.
- It consists of an array of flags, each of which describes a possible process state.
- In the current Linux version, these states are mutually exclusive, and hence exactly one flag of state always is set; the remaining flags are cleared.
- The following are the possible process states:
  - TASK_RUNNING
  - TASK_INTERRUPTIBLE
  - TASK_UNINTERRUPTIBLE
  - TASK_STOPPED
  - TASK_TRACED
  - EXIT_ZOMBIE
  - EXIT_DEAD

PROCESS STATES

- **TASK_RUNNING**
  - **The process is either executing on a CPU or waiting to be executed.**
- **TASK_INTERRUPTIBLE**
  - **The process is suspended (sleeping) until some condition becomes true. Raising a hardware interrupt, releasing a system resource the process is waiting for, or delivering a signal are examples of conditions that might wake up the process (put its state back to TASK_RUNNING).**

PROCESS STATES

- **TASK_UNINTERRUPTIBLE**

- **Like TASK_INTERRUPTIBLE, except that delivering a signal to the sleeping process leaves its state unchanged.**
- **This process state is seldom used. It is valuable, however, under certain specific conditions in which a process must wait until a given event occurs without being interrupted.**
- **For instance, this state maybe used when a process opens a device file and the corresponding device driver starts probing for a corresponding hardware device.**
- **The device driver must not be interrupted until the probing is complete, or the hardware device could be left in an unpredictable state.**

PROCESS STATES

- **TASK_STOPPED**
  - **Process execution has been stopped; the process enters this state after receiving a SIGSTOP, SIGTSTP, SIGTTIN, or SIGTTOU signal.**
- **TASK_TRACED**
  - **Process execution has been stopped by a debugger.**
  - **When a process is being monitored by another (such as when a debugger executes a ptrace( ) system call to monitor a test program), each signal may put the process in the TASK_TRACED state.**
  - **Two additional states of the process can be stored both in the state field and in the exit_state field of the process descriptor; as the field name suggests, a process reaches one of these two states only when its execution is terminated:**
    - **EXIT_ZOMBIE**
    - **EXIT_DEAD**

PROCESS STATES

- **EXIT_ZOMBIE**
  - **Process execution is terminated, but the parent process has not yet issued a wait4( ) or waitpid( ) system call to return information about the dead process.**
  - **Before the wait( )-like call is issued, the kernel cannot discard the data contained in the dead process descriptor because the parent might need it.**
  - **There are other wait( ) -like library functions, such as wait3( ) and wait( ), but in Linux they are implemented by means of the wait4( ) and waitpid( ) system calls.**
- **EXIT_DEAD**
  - **The final state: the process is being removed by the system because the parent process has just issued a wait4( ) or waitpid( ) system call for it.**
  - **Changing its state from EXIT_ZOMBIE to EXIT_DEAD avoids race conditions due to other threads of execution that execute wait( )-like calls on the same process.**

PROCESS STATES

- The value of the state field is usually set with a simple assignment.

- For instance:

  p->state = TASK_RUNNING;

- The kernel also uses the set_task_state and set_current_state macros:
  - they set the state of a specified process and of the process currently executed, respectively.

IDENTIFYING A PROCESS

- As a general rule, each execution context that can be independently scheduled must have its own process descriptor.
- Therefore, even lightweight processes, which share a large portion of their kernel data structures, have their own task_struct structures.
- The strict one-to-one correspondence between the process and process descriptor makes the 32-bit address of the task_struct structure a useful means for the kernel to identify processes.
- These addresses are referred to as process descriptor pointers

IDENTIFYING A PROCESS

- Most of the references to processes that the kernel makes are through process descriptor pointers.
- On the other hand, Unix-like operating systems allow users to identify processes by means of a number called the Process ID (or PID), which is stored in the pid field of the process descriptor.
- PIDs are numbered sequentially: the PID of a newly created process is normally the PID of the previously created process increased by one.

IDENTIFYING A PROCESS

- **There is an upper limit on the PID values**
- **When the kernel reaches such limit, it must start recycling the lower, unused PIDs.**
- **By default, the maximum PID number is 32,767 (PID_MAX_DEFAULT - 1); (1 Page of size 4KB allocated)**
- **The system administrator may reduce this limit by writing a smaller value into the**

  /proc /sys/kernel/pid_max file

  (/proc is the mount point of a special filesystem)
- **In 64-bit architectures, the system administrator can enlarge the maximum PID number up to 4,194,303.**

  (128 pages can be used → 128x4kx8= 128x4096x8 =4,194,304)

IDENTIFYING A PROCESS

- When recycling PID numbers, the kernel must manage a pidmap_array bitmap that denotes which are the PIDs currently assigned and which are the free ones.
- Because a page frame contains 32,768 bits, in 32-bit architectures the pidmap_array bitmap is stored in a single page.
- In 64-bit architectures, however, additional pages can be added to the bitmap when the kernel assigns a PID number too large for the current bitmap size.

IDENTIFYING A PROCESS

- Linux associates a different PID with each process or lightweight process in the system.
- This approach allows the maximum flexibility, because every execution context in the system can be uniquely identified.

- On the other hand, Unix programmers expect threads in the same group to have a common PID.
- For instance, it should be possible to a send a signal specifying a PID that affects all threads in the group.

IDENTIFYING A PROCESS

- In fact, the POSIX(Portable Operating System Interface) 1003.1c standard states that all threads of a multithreaded application must have the same PID.
- To comply with this standard, Linux makes use of thread groups.
- The identifier shared by the threads is the PID of the thread group leader , that is, the PID of the first lightweight process in the group; it is stored in the tgid field of the process descriptors.
- The getpid( ) system call returns the value of tgid relative to the current process instead of the value of pid, so all the threads of a multithreaded application share the same identifier.
- Most processes belong to a thread group consisting of a single member; as thread group leaders, they have the tgid field equal to the pid field, thus the getpid( ) system call works as usual for this kind of process.

PROCESS DESCRIPTORS HANDLING

- Processes are dynamic entities whose lifetimes range from a few milliseconds to months.
- Thus, the kernel must be able to handle many processes at the same time, and process descriptors are stored in dynamic memory rather than in the memory area permanently assigned to the kernel.

PROCESS DESCRIPTORS HANDLING

- For each process, Linux packs two different data structures in a single per-process memory area: a small data structure linked to the process descriptor, namely the thread_info structure, and the Kernel Mode process stack.
- The length of this memory area is usually 8,192 bytes (two page frames).
- For reasons of efficiency the kernel stores the 8-KB memory area in two consecutive page frames with the first page frame aligned to a multiple of 2^{13}; this may turn out to be a problem when little dynamic memory is available, because the free memory may become highly fragmented
- Therefore, in the 80x86 architecture the kernel can be configured at compilation time so that the memory area including stack and thread_info structure spans a single page frame (4,096 bytes).

PROCESS DESCRIPTORS HANDLING

- A process in Kernel Mode accesses a stack contained in the kernel data segment, which is different from the stack used by the process in User Mode.
- Because kernel control paths make little use of the stack, only a few thousand bytes of kernel stack are required.
- Therefore, 8 KB is ample space for the stack and the tHRead_info structure.
- However, when stack and thread_info structure are contained in a single page frame, the kernel uses a few additional stacks to avoid the overflows caused by deeply nested interrupts and exceptions

PROCESS DESCRIPTORS HANDLING

Storing the thread_info structure and the process kernel stack in two page frames

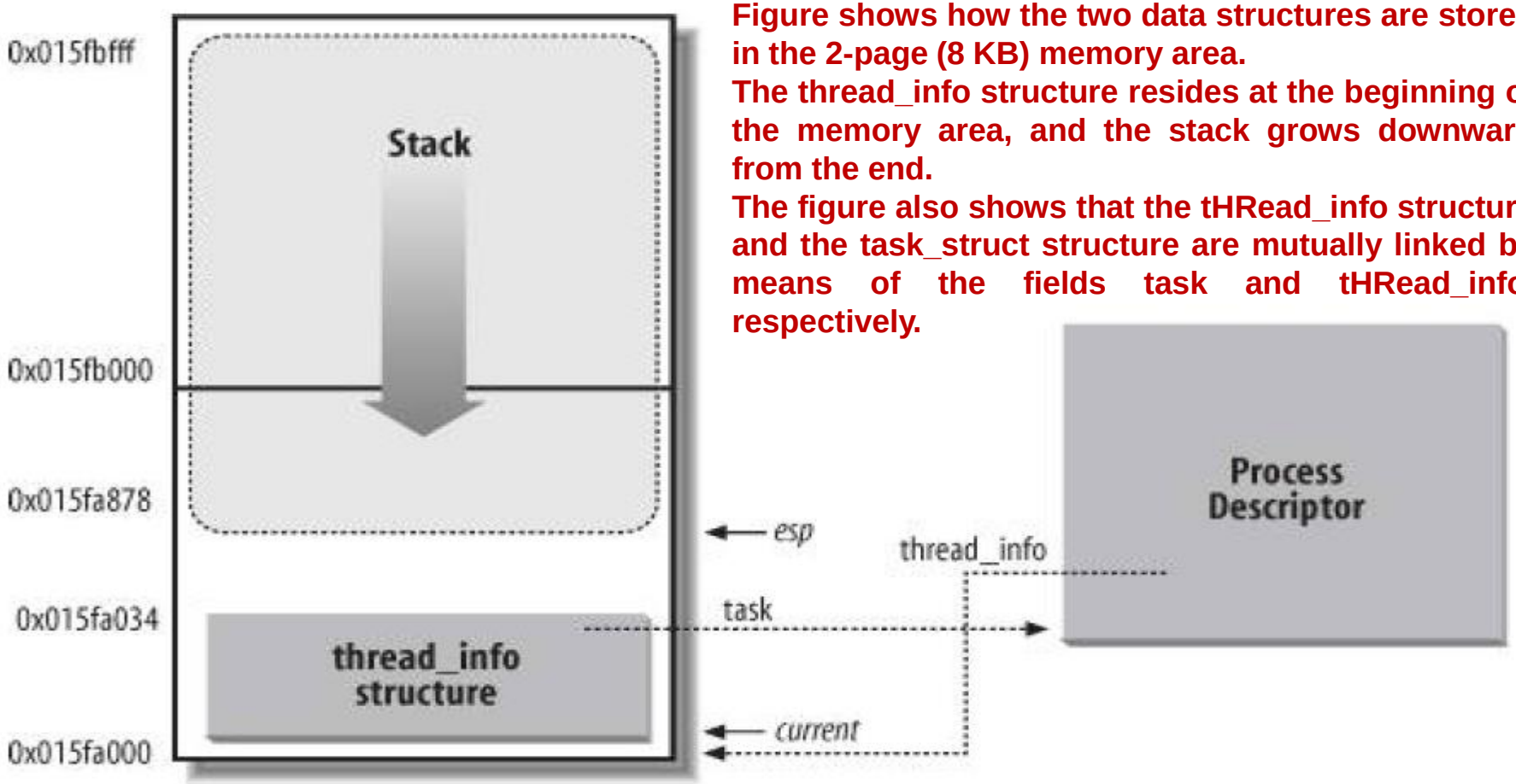


Figure shows how the two data structures are stored in the 2-page (8 KB) memory area.
The thread_info structure resides at the beginning of the memory area, and the stack grows downward from the end.
The figure also shows that the tHRead_info structure and the task_struct structure are mutually linked by means of the fields task and tHRead_info, respectively.

The tHRead_info structure shown in Figure is stored starting at address 0x015fa000, and the stack is stored starting at address 0x015fc000.
The value of the esp register points to the current top of the stack at 0x015fa878.
The kernel uses the alloc_thread_info and free_thread_info macros to allocate and release the memory area storing a thread_info structure and a kernel stack.

PROCESS DESCRIPTORS HANDLING

- The esp register is the CPU stack pointer, which is used to address the stack's top location.
- On 80x86 systems, the stack starts at the end and grows toward the beginning of the memory area.
- Right after switching from User Mode to Kernel Mode, the kernel stack of a process is always empty, and therefore the esp register points to the byte immediately following the stack.
- The value of the esp is decreased as soon as data is written into the stack.
- Because the thread_info structure is 52 bytes long, the kernel stack can expand up to 8,140 bytes.

IDENTIFYING THE CURRENT PROCESS

- **The close association between the thread_info structure and the Kernel Mode stack offers a key benefit in terms of efficiency:**
- **The kernel can easily obtain the address of the thread_info structure of the process currently running on a CPU from the value of the esp register.**
- **In fact, if the thread_union structure is 8 KB (2^{13} bytes) long, the kernel masks out the 13 least significant bits of esp to obtain the base address of the thread_info structure;**
- **on the other hand, if the thread_union structure is 4 KB long, the kernel masks out the 12 least significant bits of esp.**

- **Most often the kernel needs the address of the process descriptor rather than the address of the thread_info structure.**
- **To get the process descriptor pointer of the process currently running on a CPU, the kernel makes use of the current macro, which is essentially equivalent to current_thread_info( )->task**
- **The current macro often appears in kernel code as a prefix to fields of the process descriptor.**
- **For example, current->pid returns the process ID of the process currently running on the CPU.**

IDENTIFYING THE CURRENT PROCESS

- Another advantage of storing the process descriptor with the stack emerges on multiprocessor systems:
- The correct current process for each hardware processor can be derived just by checking the stack
- Earlier versions of Linux did not store the kernel stack and the process descriptor together.
- Instead, they were forced to introduce a global static variable called current to identify the process descriptor of the running process.
- On multiprocessor systems, it was necessary to define current as an array one element for each available CPU.

PROCESS LIST

- In Linux PROCESS LIST is implemented as a doubly linked list, a list that links together all existing process descriptors.
- Each task_struct structure includes a tasks field of type list_head whose prev and next fields point, respectively, to the previous and to the next task_struct element.
- The head of the process list is the init_task task_struct descriptor; it is the process descriptor of the so-called process 0 or swapper
- The tasks->prev field of init_task points to the tasks field of the process descriptor inserted last in the list.
- The SET_LINKS and REMOVE_LINKS macros are used to insert and to remove a process descriptor in the process list, respectively.
- These macros also take care of the parenthood relationship of the process

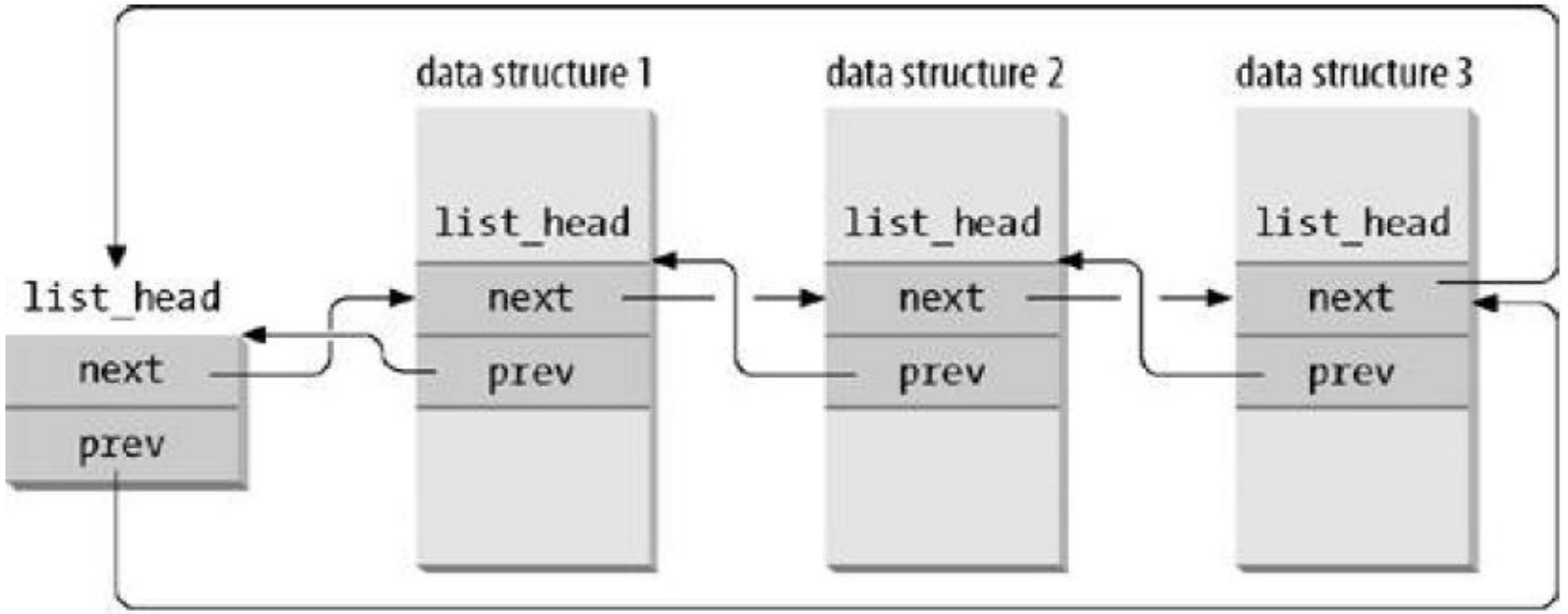
Doubly linked lists

- **Role of special data structures that implement doubly linked lists**

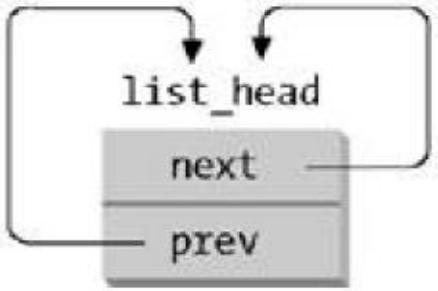
- **For each list, a set of primitive operations must be implemented: initializing the list, inserting and deleting an element, scanning the list, and so on.**
- **Linux kernel defines the list_head data structure, whose only fields next and prev represent the forward and back pointers of a generic doubly linked list element, respectively.**
- **It is important to note, however, that the pointers in a list_head field store the addresses of other list_head fields rather than the addresses of the whole data structures in which the list_head structure is included;**
- **A new list is created by using the LIST_HEAD(list_name) macro.**
- **It declares a new variable named list_name of type list_head, which is a dummy first element that acts as a placeholder for the head of the new list, and initializes the prev and next fields of the list_head data structure so as to point to the list_name variable itself**

Doubly linked lists

Doubly linked lists built with list_head data structures



(a) a doubly linked listed with three elements



(b) an empty doubly linked list

LIST HANDLING FUNCTIONS

- The Linux kernel sports another kind of doubly linked list, which mainly differs from a list_head list because it is not circular; it is mainly used for hash tables, where space is important, and finding the the last element in constant time is not.
- The list head is stored in an hlist_head data structure, which is simply a pointer to the first element in the list (NULL if the list is empty).
- Each element is represented by an hlist_node data structure, which includes a pointer next to the next element, and a pointer pprev to the next field of the previous element.
- Because the list is not circular, the pprev field of the first element and the next field of the last element are set to NULL.
- The list can be handled by means of several helper functions and macros :

  hlist_add_head( ), hlist_del( ), hlist_empty( ), hlist_entry, hlist_for_each_entry, and so on.

TASK_RUNNING processes

- **The lists of TASK_RUNNING processes**

- **When looking for a new process to run on a CPU, the kernel has to consider only the runnable processes (that is, the processes in the TASK_RUNNING state).**
- **Earlier Linux versions put all runnable processes in the same list called runqueue.**
- **Because it would be too costly to maintain the list ordered according to process priorities, the earlier schedulers were compelled to scan the whole list in order to select the "best" runnable process.**
- **Linux implements the runqueue differently.**
- **The aim is to allow the scheduler to select the best runnable process in constant time, independently of the number of runnable processes.**

TASK_RUNNING processes

- The trick used to achieve the scheduler speedup consists of splitting the runqueue in many lists of runnable processes, one list per process priority.
- Each task_struct descriptor includes a run_list field of type list_head.
- If the process priority is equal to k (a value ranging between 0 and 139), the run_list field links the process descriptor into the list of runnable processes having priority k.
- Furthermore, on a multiprocessor system, each CPU has its own runqueue, that is, its own set of lists of processes.
- This is a classic example of making a data structures more complex to improve performance:
  - To make scheduler operations more efficient, the runqueue list has been split into 140 different lists!

- The kernel must preserve a lot of data for every runqueue in the system.
- The main data structures of a runqueue are the lists of process descriptors belonging to the runqueue;
- All these lists are implemented by a single prio_array_t data structure, whose fields are shown in Table

Table : The fields of the prio_array_t data structure

Type	Field	Description
Int	nr_active	The number of process descriptors linked into the lists
unsigned long [5]	bitmap	A priority bitmap: each flag is set if and only if the corresponding priority list is not empty
struct list_head [140]	queue	The 140 heads of the priority lists

enqueue_task(prio,array) function inserts a process descriptor into a runqueue list;
The prio field of the process descriptor stores the dynamic priority of the process, while the array field is a pointer to the prio_array_t data structure of its current runqueue.

dequeue_task(p,array) function removes a process descriptor from a runqueue list.

Relationships Among Processes

- **Relationships Among Processes**

- **Processes created by a program have a parent/child relationship.**
- **When a process creates multiple children , these children have sibling relationships.**
- **Several fields must be introduced in a process descriptor to represent these relationships; they are listed in Table with respect to a given process P.**
- **Processes 0 and 1 are created by the kernel;**
- **Process 1 (init) is the ancestor of all other processes.**

Relationships Among Processes

- **Relationships Among Processes**
  - **Fields of a process descriptor used to express parenthood relationships**
- **real_parent**
  - **Points to the process descriptor of the process that created P or to the descriptor of process 1 (init) if the parent process no longer exists. (Therefore, when a user starts a background process and exits the shell, the background process becomes the child of init.)**
- **parent**
  - **Points to the current parent of P (this is the process that must be signaled when the child process terminates); its value usually coincides with that of real_parent.**
  - **It may occasionally differ, such as when another process issues a ptrace( ) system call requesting that it be allowed to monitor P**
- **children**
  - **The head of the list containing all children created by P.**
- **sibling**
  - **The pointers to the next and previous elements in the list of the sibling processes, those that have the same parent as P.**

PROCESSES

Parenthood relationships among five processes

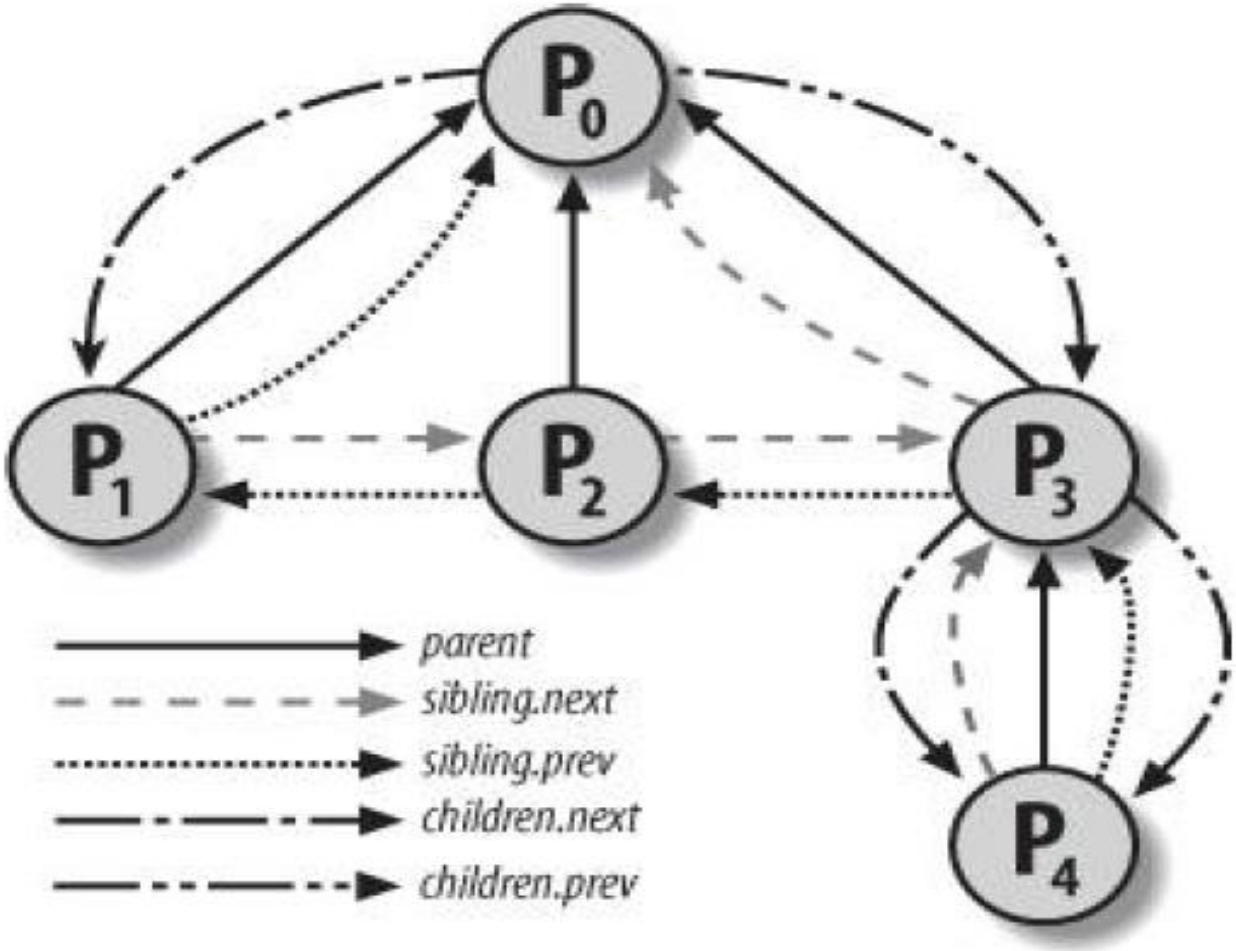


Figure illustrates the parent and sibling relationships of a group of processes. Process P0 successively created P1, P2, and P3. Process P3, in turn, created process P4.

PROCESSES

- Furthermore, there exist other relationships among processes: a process can be a leader of a process group or of a login session it can be a leader of a thread group, and it can also trace the execution of other processes.
- Table lists the fields of the process descriptor that establish these relationships between a process P and the other processes

Table : The fields of the process descriptor that establish non-parenthood relationships

Field name	Description
group_leader	Process descriptor pointer of the group leader of P
signal->pgrp	PID of the group leader of P
tgid	PID of the thread group leader of P
signal->session	PID of the login session leader of P
ptrace_children	The head of a list containing all children of P being traced by a debugger
ptrace_list	The pointers to the next and previous elements in the real parent's list of traced processes (used when P is being traced)

pidhash table and chained lists

- pidhash table and chained lists

- In several circumstances, the kernel must be able to derive the process descriptor pointer corresponding to a PID.
- Scanning the process list sequentially and checking the pid fields of the process descriptors is feasible but rather inefficient.
- To speed up the search, four hash tables have been introduced.
- Why multiple hash tables?
- Simply because the process descriptor includes fields that represent different types of PID, and each type of PID requires its own hash table.

pidhash table and chained lists

- pidhash table and chained lists
  - Table : The four hash tables and corresponding fields in the process descriptor

Hash table type	Field name	Description
PIDTYPE_PID	pid	PID of the process
PIDTYPE_TGID	tgid	PID of thread group leader process
PIDTYPE_PGID	pgrp	PID of the group leader process
PIDTYPE_SID	session	PID of the session leader process

- The four hash tables are dynamically allocated during the kernel initialization phase, and their addresses are stored in the pid_hash array.
- The size of a single hash table depends on the amount of available RAM; for example, for systems having 512 MB of RAM, each hash table is stored in four page frames and includes 2,048 entries.

pidhash table and chained lists

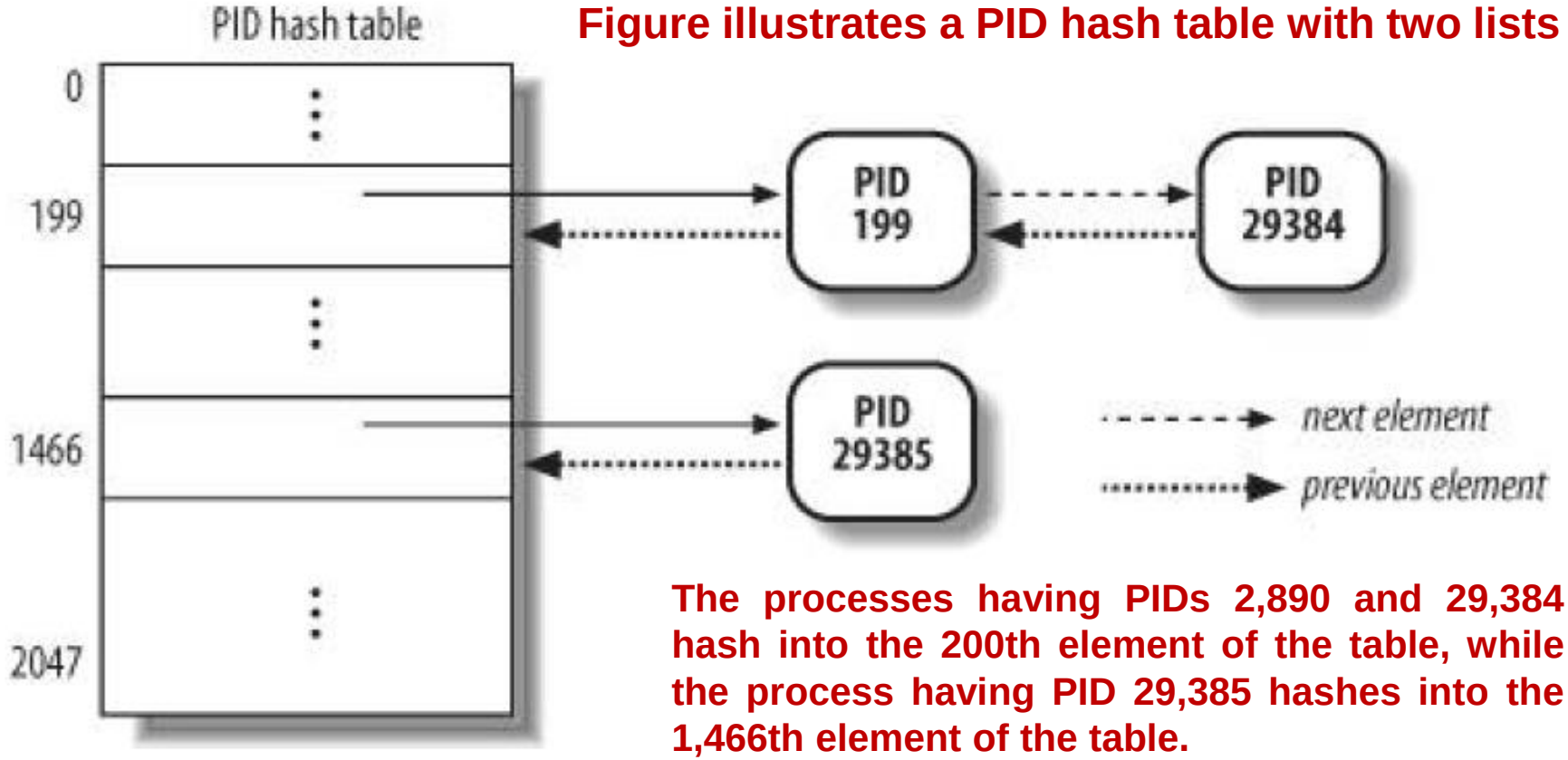
- Linux uses chaining to handle colliding PIDs; each table entry is the head of a doubly linked list of colliding process descriptors
- Figure illustrates a PID hash table with two lists.
- The processes having PIDs 199 and 29,384 hash into the 199th element of the table, while the process having PID 29,385 hashes into the 1,466th element of the table.
- Hashing with chaining is preferable to a linear transformation from PIDs to table indexes because at any given instance, the number of processes in the system is usually far below 32,768 (the maximum number of allowed PIDs).
- It would be a waste of storage to define a table consisting of 32,768 entries, if, at any given instance, most such entries are unused.

pidhash table and chained lists

Linux uses chaining to handle colliding PIDs; each table entry is the head of a doubly linked list of colliding process descriptors

A simple PID hash table and chained lists

Figure illustrates a PID hash table with two lists



The processes having PIDs 2,890 and 29,384 hash into the 200th element of the table, while the process having PID 29,385 hashes into the 1,466th element of the table.

Hashing with chaining is preferable to a linear transformation from PIDs to table indexes because at any given instance, the number of processes in the system is usually far below 32,768 (the maximum number of allowed PIDs).

pidhash table and chained lists

- The data structures used in the PID hash tables are quite sophisticated, because they must keep track of the relationships between the processes.
- As an example, suppose that the kernel must retrieve all processes belonging to a given thread group, that is, all processes whose tgid field is equal to a given number.
- Looking in the hash table for the given thread group number returns just one process descriptor, that is, the descriptor of the thread group leader.
- To quickly retrieve the other processes in the group, the kernel must maintain a list of processes for each thread group.
- The same situation arises when looking for the processes belonging to a given login session or belonging to a given process group.

pidhash table and chained lists

- **The PID hash tables' data structures solve all these problems, because they allow the definition of a list of processes for any PID number included in a hash table.**

- **The core data structure is an array of four pid structures embedded in the pids field of the process descriptor; the fields of the pid structure are shown in Table**

The fields of the pid data structures

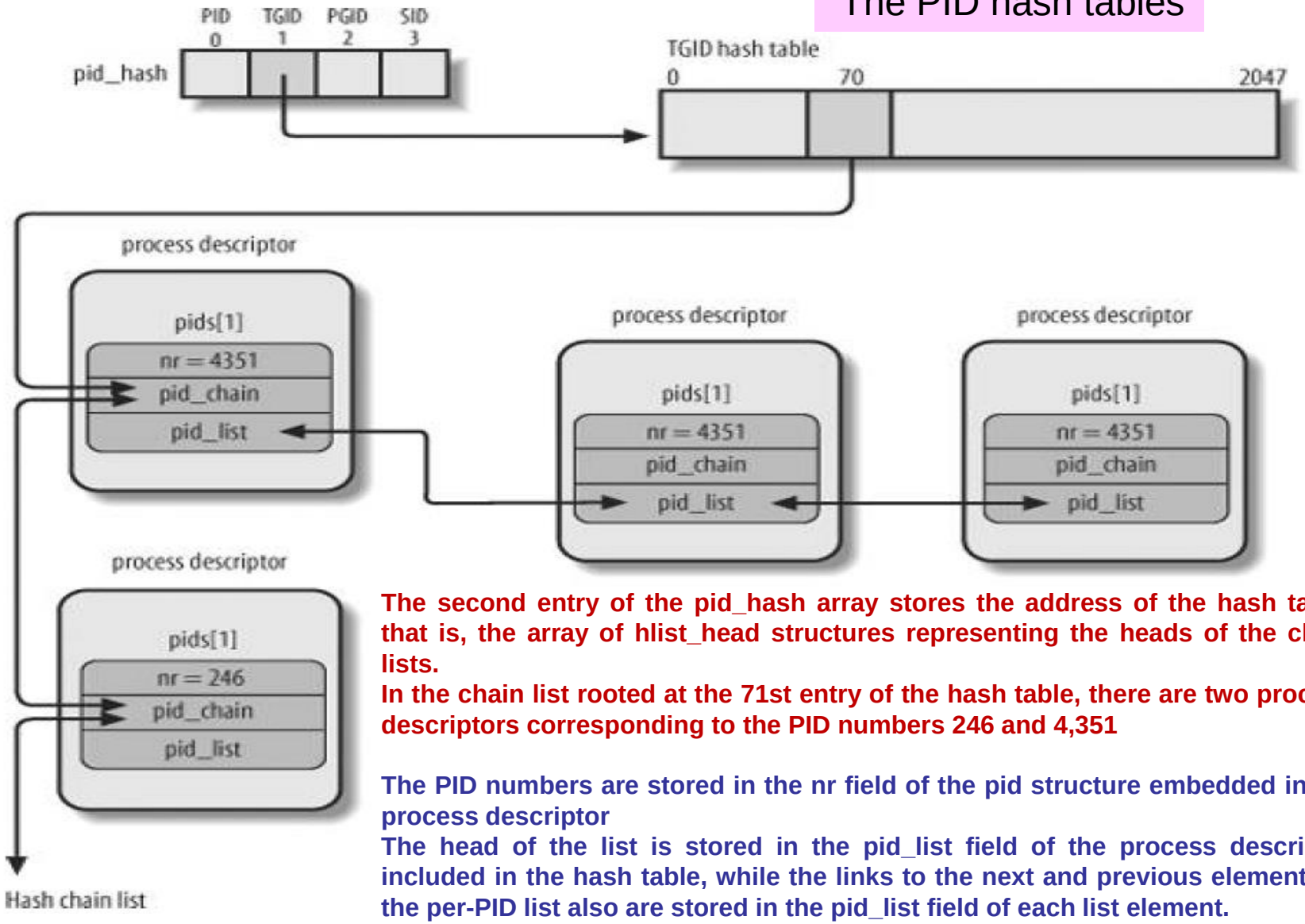
Type	Name	Description
int	nr	The PID number
struct hlist_node	pid_chain	The links to the next and previous elements in the hash chain list
struct list_head	pid_list	The head of the per-PID list

pidhash table and chained lists

- Figure shows an example based on the PIDTYPE_TGID hash table
- The second entry of the pid_hash array stores the address of the hash table, that is, the array of hlist_head structures representing the heads of the chain lists.
- In the chain list rooted at the 71st entry of the hash table, there are two process descriptors corresponding to the PID numbers 246 and 4,351 (double-arrow lines represent a couple of forward and backward pointers).
- The PID numbers are stored in the nr field of the pid structure embedded in the process descriptor (by the way, because the thread group number coincides with the PID of its leader, these numbers also are stored in the pid field of the process descriptors).
- Let us consider the per-PID list of the thread group 4,351: the head of the list is stored in the pid_list field of the process descriptor included in the hash table, while the links to the next and previous elements of the per-PID list also are stored in the pid_list field of each list element.

pidhash table and chained lists

Figure shows an example based on the PIDTYPE_TGID hash table



The second entry of the pid_hash array stores the address of the hash table, that is, the array of hlist_head structures representing the heads of the chain lists.
In the chain list rooted at the 71st entry of the hash table, there are two process descriptors corresponding to the PID numbers 246 and 4,351

The PID numbers are stored in the nr field of the pid structure embedded in the process descriptor
The head of the list is stored in the pid_list field of the process descriptor included in the hash table, while the links to the next and previous elements of the per-PID list also are stored in the pid_list field of each list element.

How Processes Are Organized

- The runqueue lists group all processes in a TASK_RUNNING state.
- When it comes to grouping processes in other states, the various states call for different types of treatment, with Linux opting for one of the choices shown in the following list.
  - Processes in a TASK_STOPPED, EXIT_ZOMBIE, or EXIT_DEAD state are not linked in specific lists.
  - There is no need to group processes in any of these three states, because stopped, zombie, and dead processes are accessed only via PID or via linked lists of the child processes for a particular parent.
  - Processes in a TASK_INTERRUPTIBLE or TASK_UNINTERRUPTIBLE state are subdivided into many classes, each of which corresponds to a specific event.
  - In this case, the process state does not provide enough information to retrieve the process quickly, so it is necessary to introduce additional lists of processes called wait queues

SOC 3010
OPERATING SYSTEM

END OF
WEEK 9 LECTURE2

PROCESSES

WISH YOU ALL THE BEST

INHA UNIVERSITY TASHKENT
FALL SEMESTER 2024

SOC 3010
OPERATING SYSTEM

CREDITS/HOURS PER WEEK : 3/3

COURSE TYPE : TECHNICAL CORE SEQUENCE

INSTRUCTOR
DR. A. R. NASEER
HEAD & PROFESSOR OF COMPUTER SCIENCE & ENGG